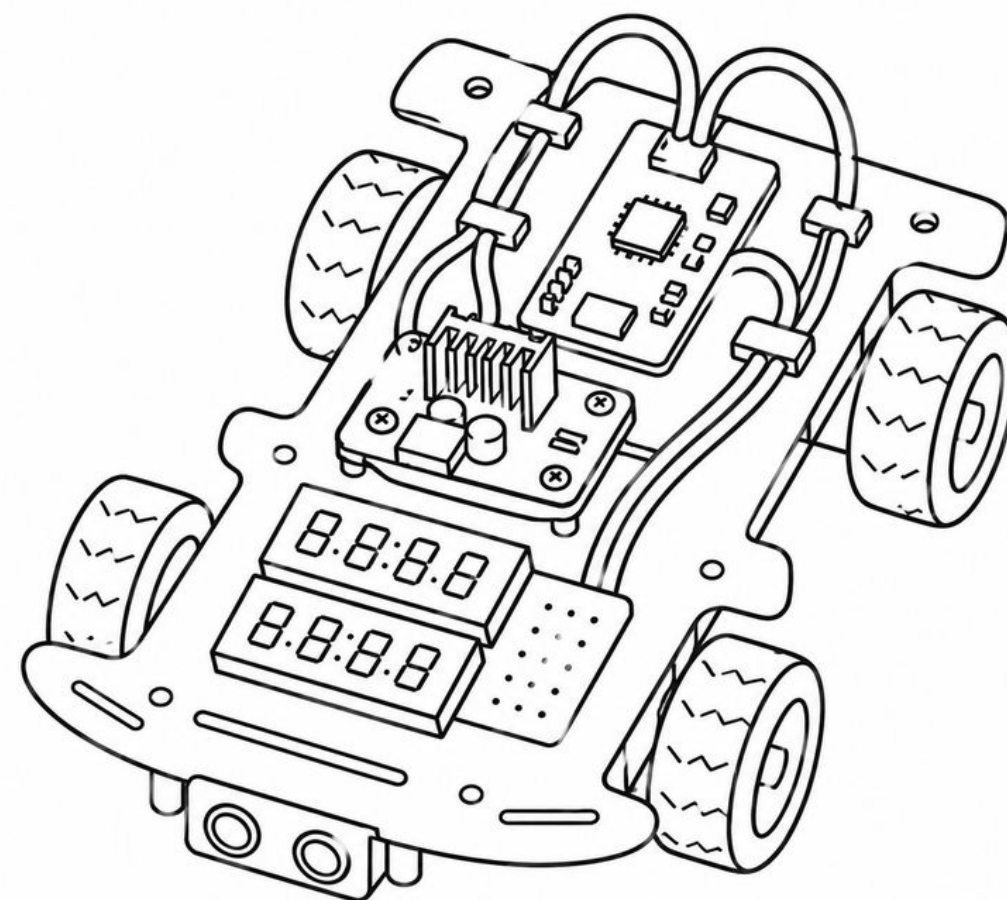


초음파센서 AVR 기반 자율주행차 프로젝트 -Auto Car

김선호, 김승환



프로젝트 목차

01

프로젝트 개요

- 자율주행차 주요 기능 및 부품 설명
- 조립 외관

02

FSM

- 상태천이도
- 세부기능
- 회로도

03

작동 영상

- 자율주행차 시연 영상

04

기능 구현

- 소프트웨어 코드 설명 및 검증
- 요소 기술

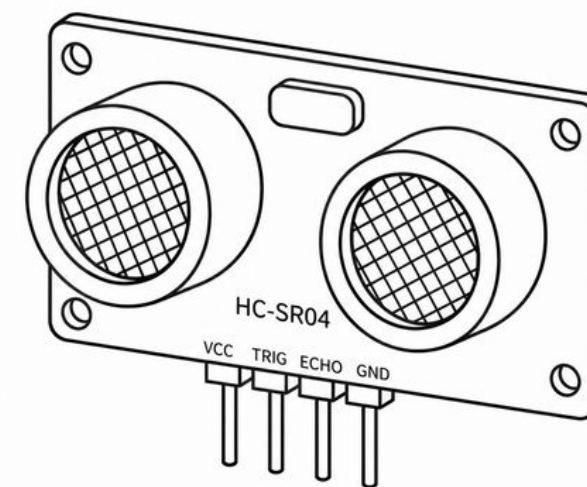
05

결론

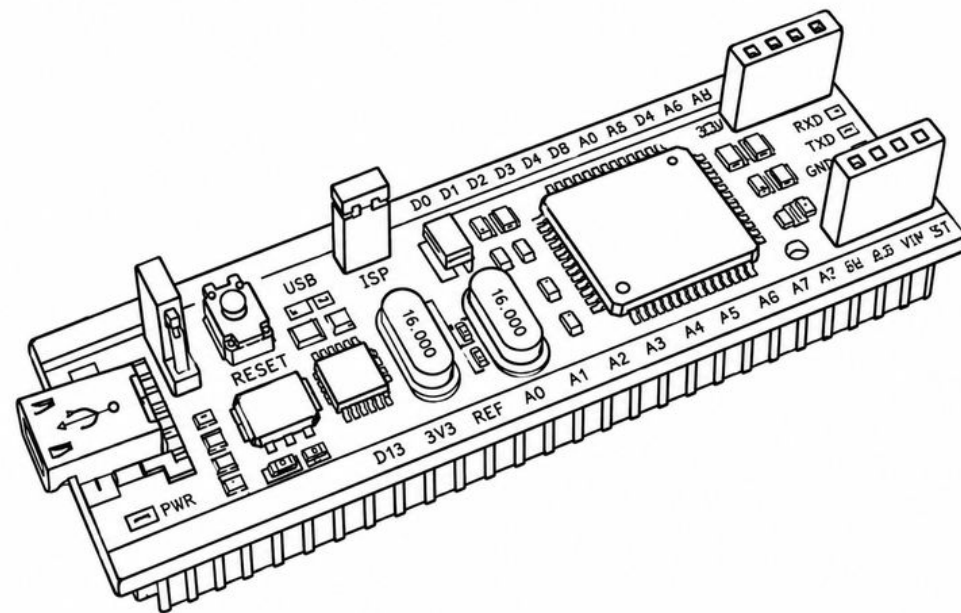
- 프로젝트 트러블 슈팅 및 느낀점



1. 프로젝트 개요



프로젝트 개요



프로젝트 목적

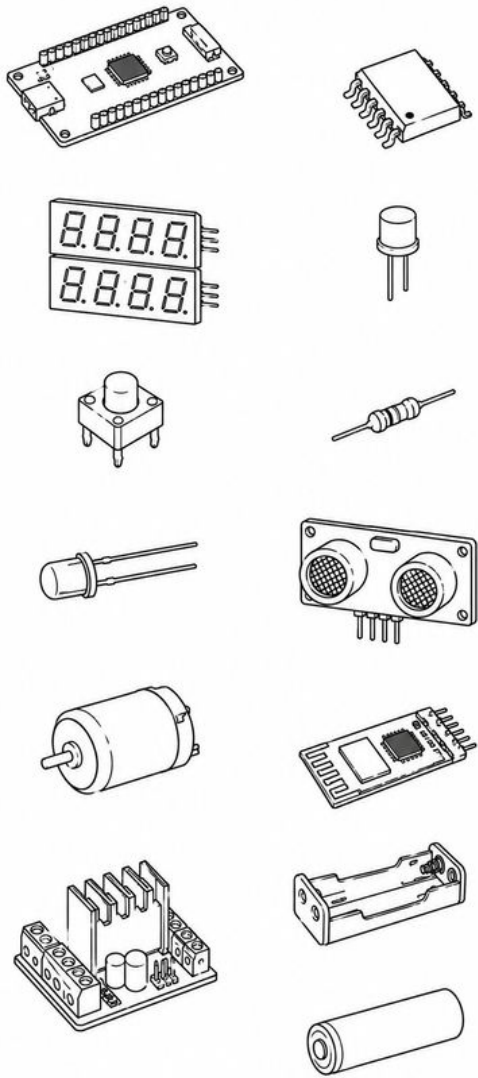
- 초음파 센서를 이용하여 전방 장애물을 감지하고 AVR MCU 기반의 FSM 제어를 통한 자율주행 차량 구현
- Bluetooth 통신을 이용한 Manual Mode와 자율주행을 위한 Auto Mode를 지원하여 임베디드 시스템 설계 능력 향상

개발 목표

- 자율 주행 구현
- FND 상태 표시
- FND 2개 동시 제어
- UART Bluetooth 통신 수동제어 구현
- PWM 모터 제어
- 거리기반 장애물 회피(Ultrasonic)

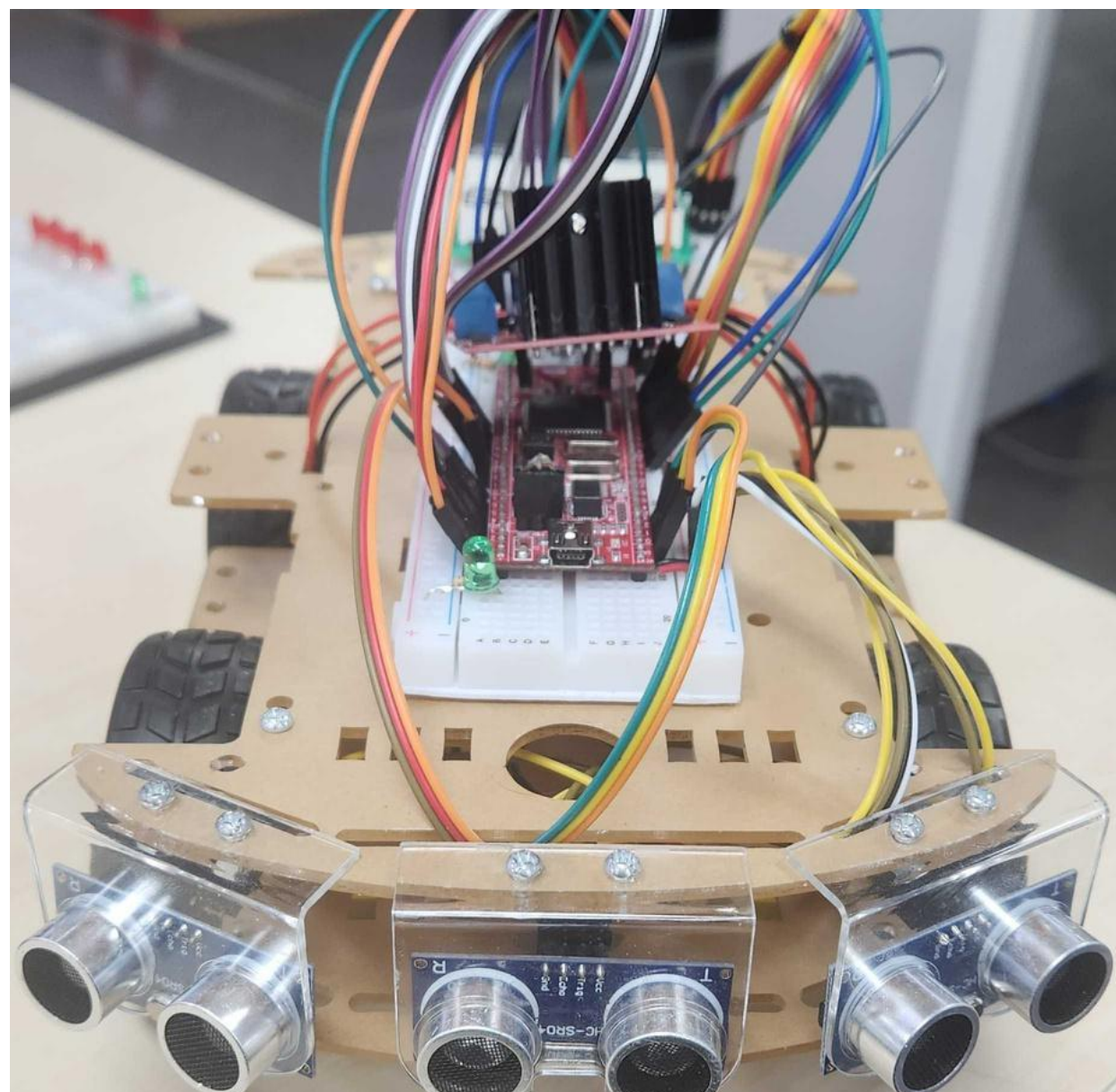
주요 부품

No	부품 명	부품 번호	용도
1	ATmega128A - Board	ATmega128A	메인 제어
2	FND	CL5642AH42	주행시간 및 기능 실행 횟수
3	BUTTON	push button switch	모드 변경
4	LED	Red LED 5mm	실행 표시
5	PWM	DG01D	자율주행차 바퀴 모터
6	Motor Control Driver	HW-095	모터 제어 모듈
7	저항(R)	resister	저항
8	초음파 센서	HC-SR04	자율주행 거리 계산
9	블루투스 모듈	ZS-040	수동모드 명령 송수신
10	배터리 홀더	ND-720	자율주행차 배터리 홀더
11	배터리	INR-18650-35E	모터 구동용 배터리

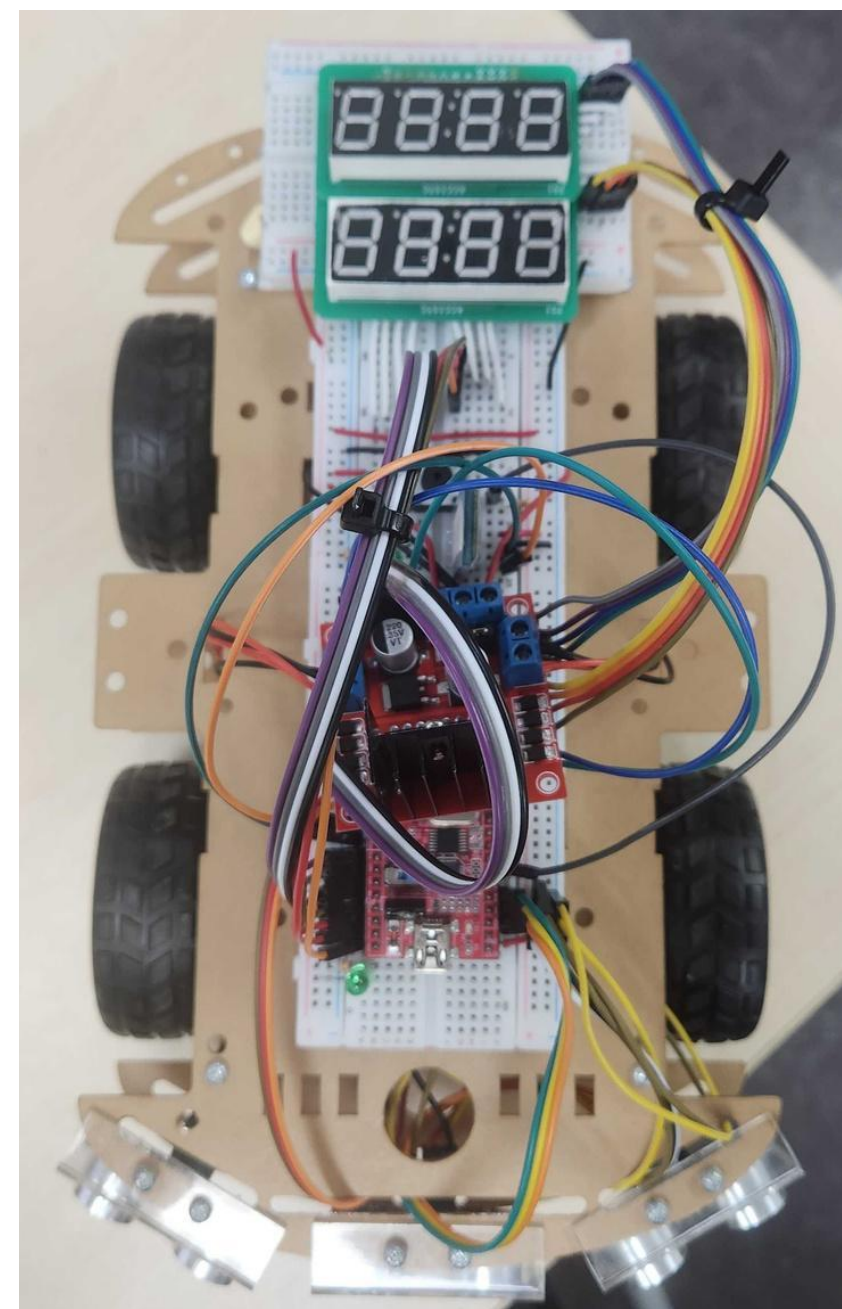


...

조립 외관



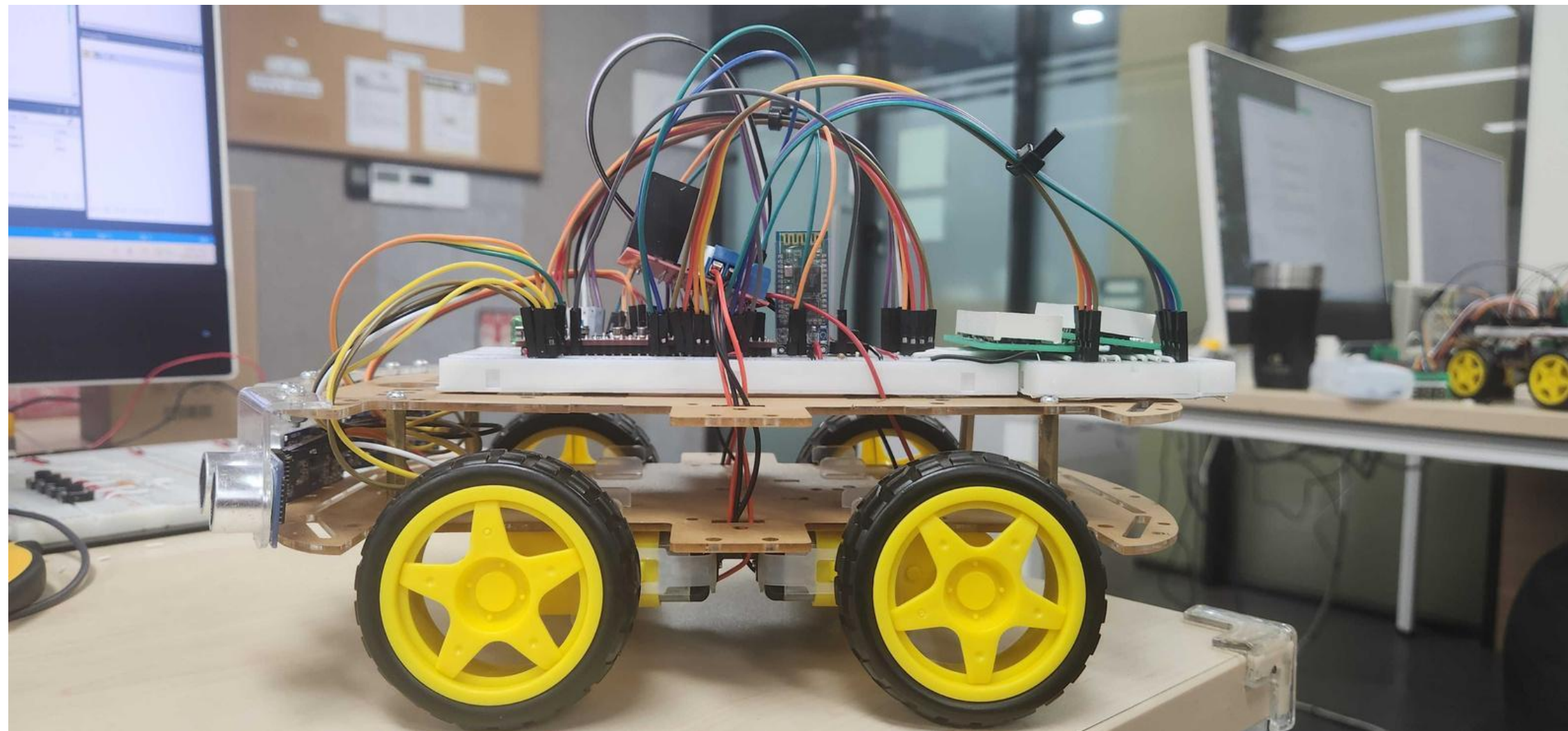
정면



위

...

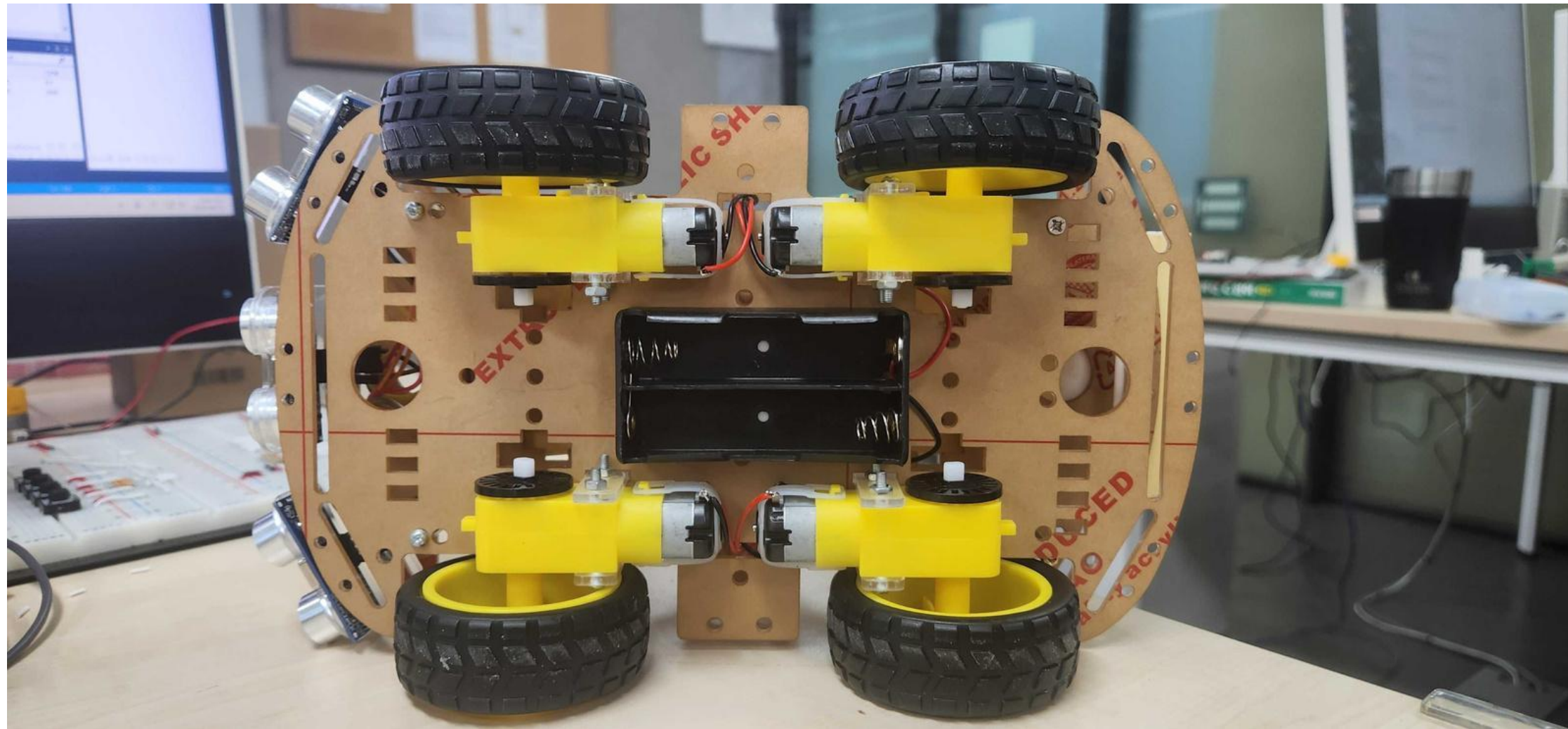
조립 외관



측면

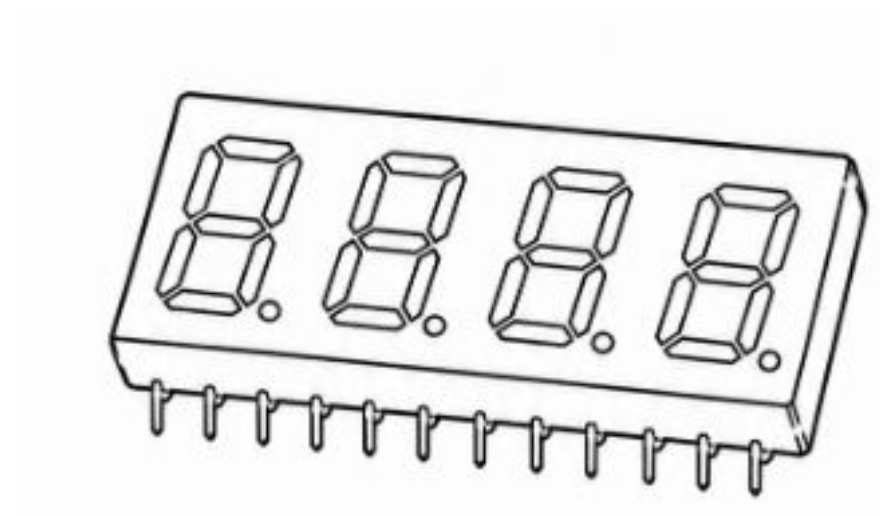
...

조립 외관

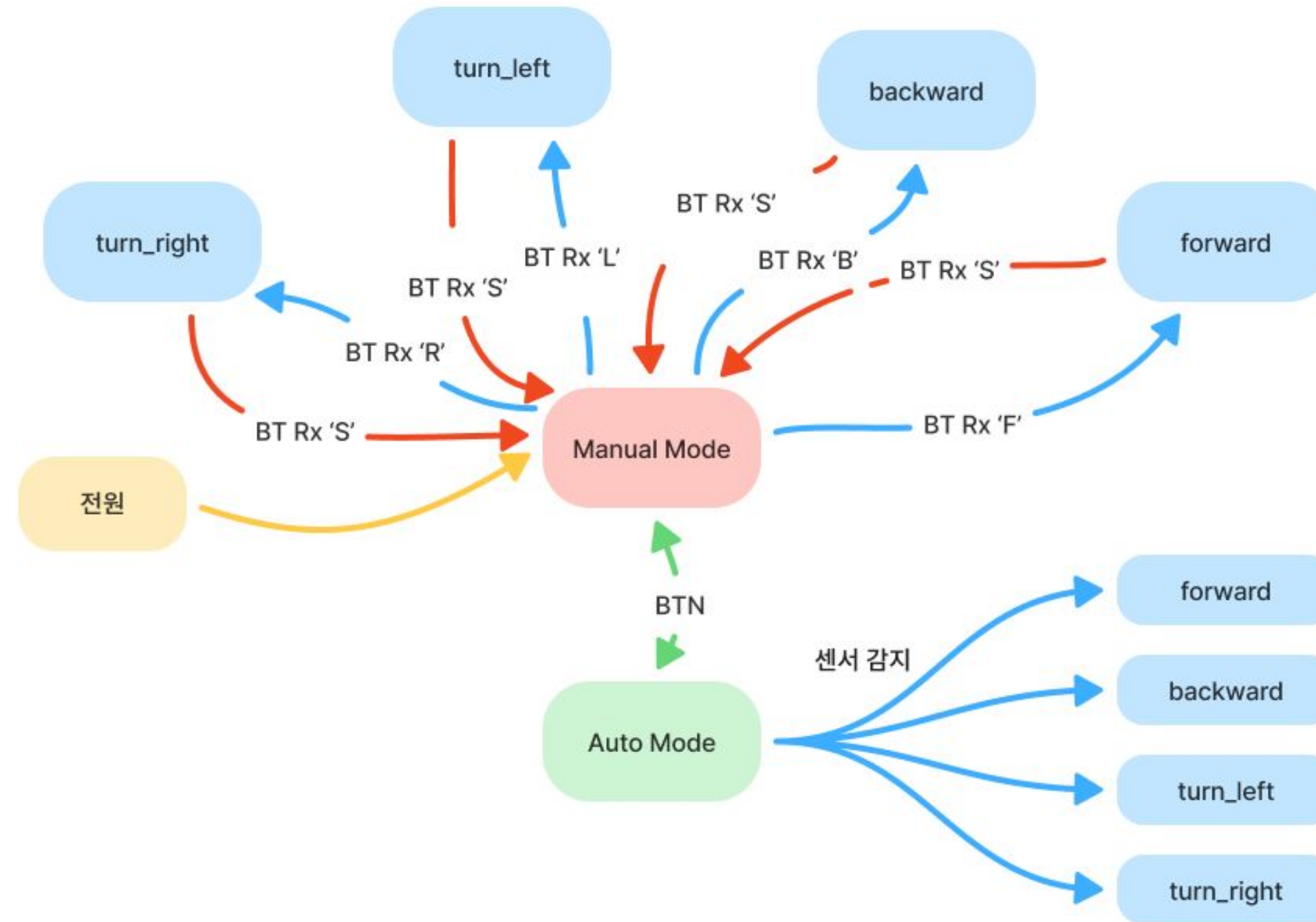


배면

2. FSM



상태천이도 (FSM, Finite State Machine)

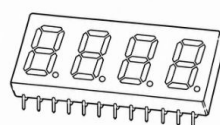


세부기능



LED

- 전원 On / Off 표시



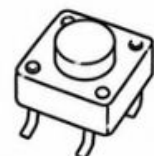
FND

- Auto Mode 작동시간 표시
- 기능 실행 횟수 표시



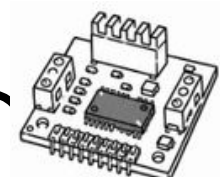
초음파 센서

- 자율주행 거리 측정



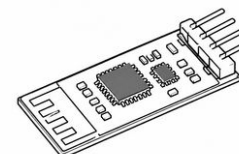
BUTTON

- Manual Mode, Auto Mode
전환



PWM

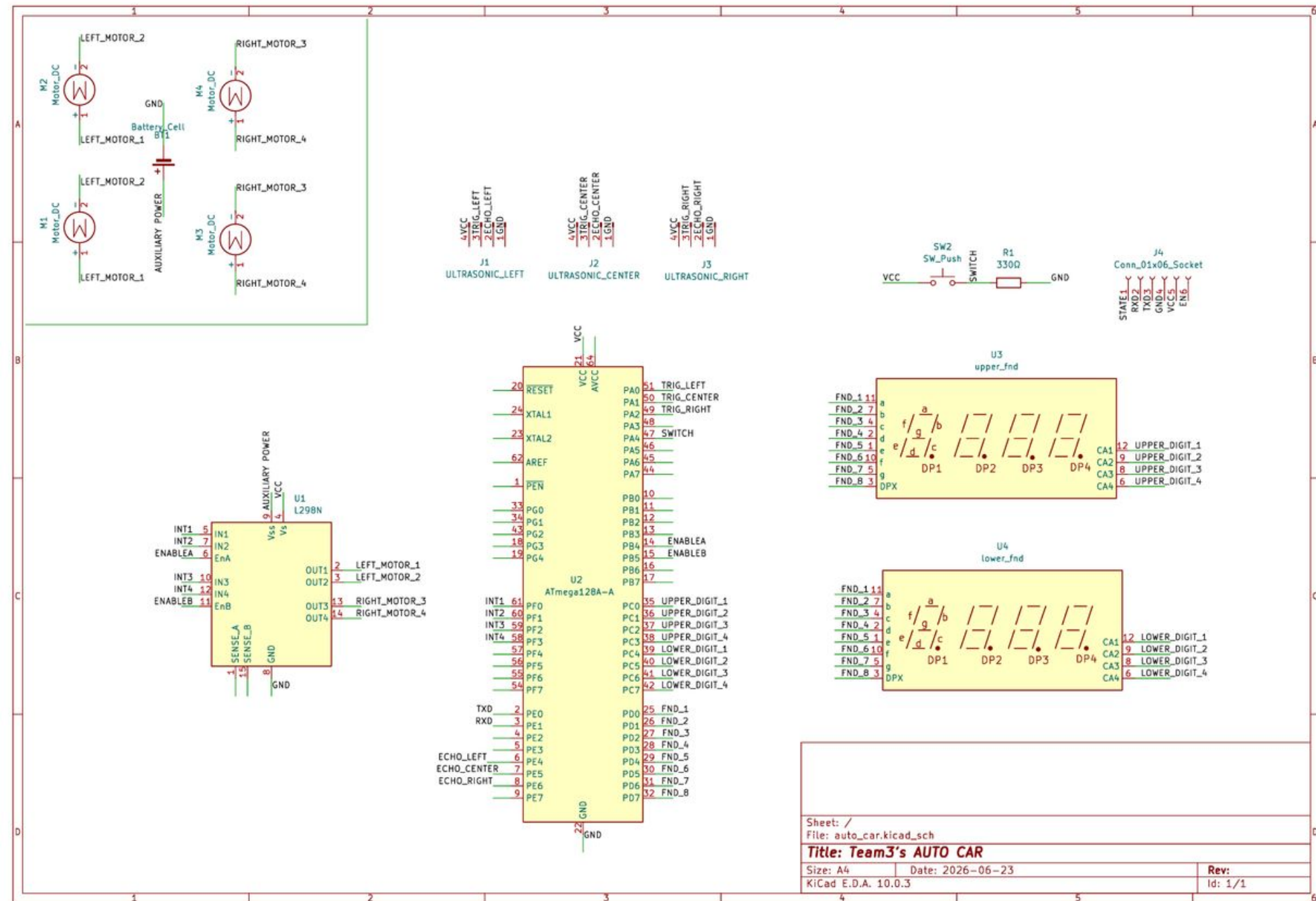
- 모터 속도 조절
- 모터 방향 조절



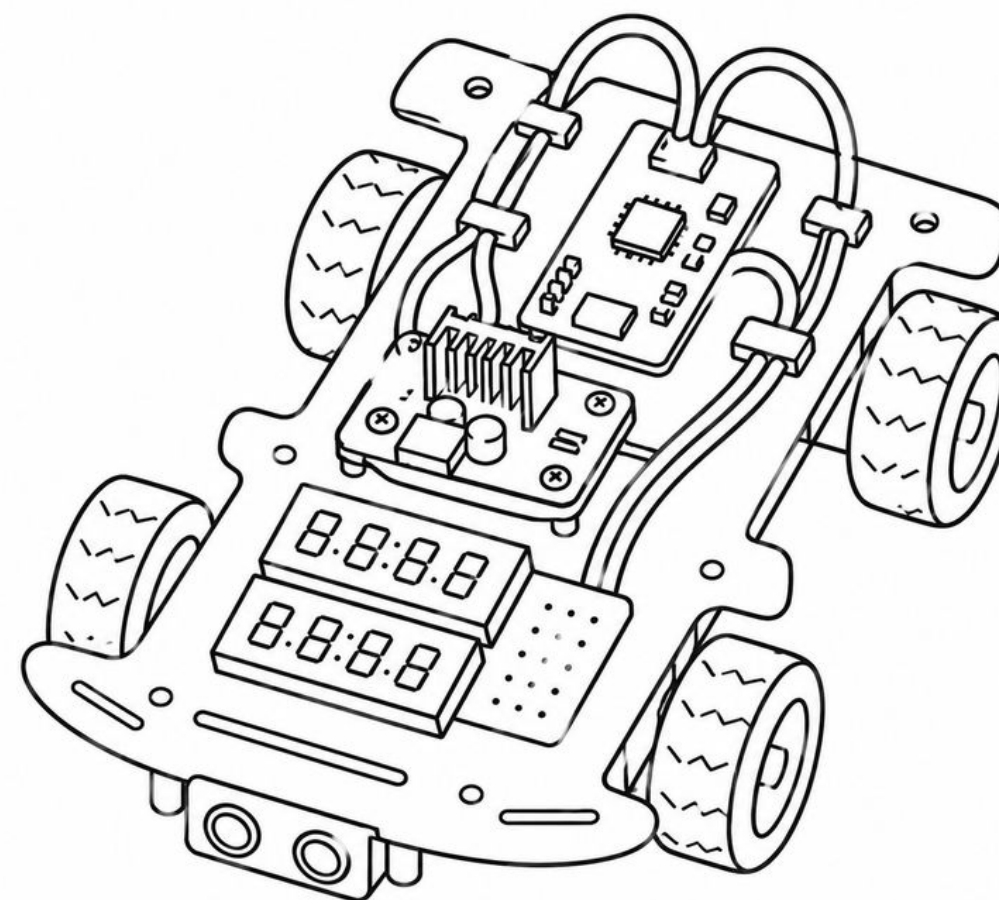
UART / BT

- Bluetooth 송수신
- INT Debug 체크

회로도(kicad)



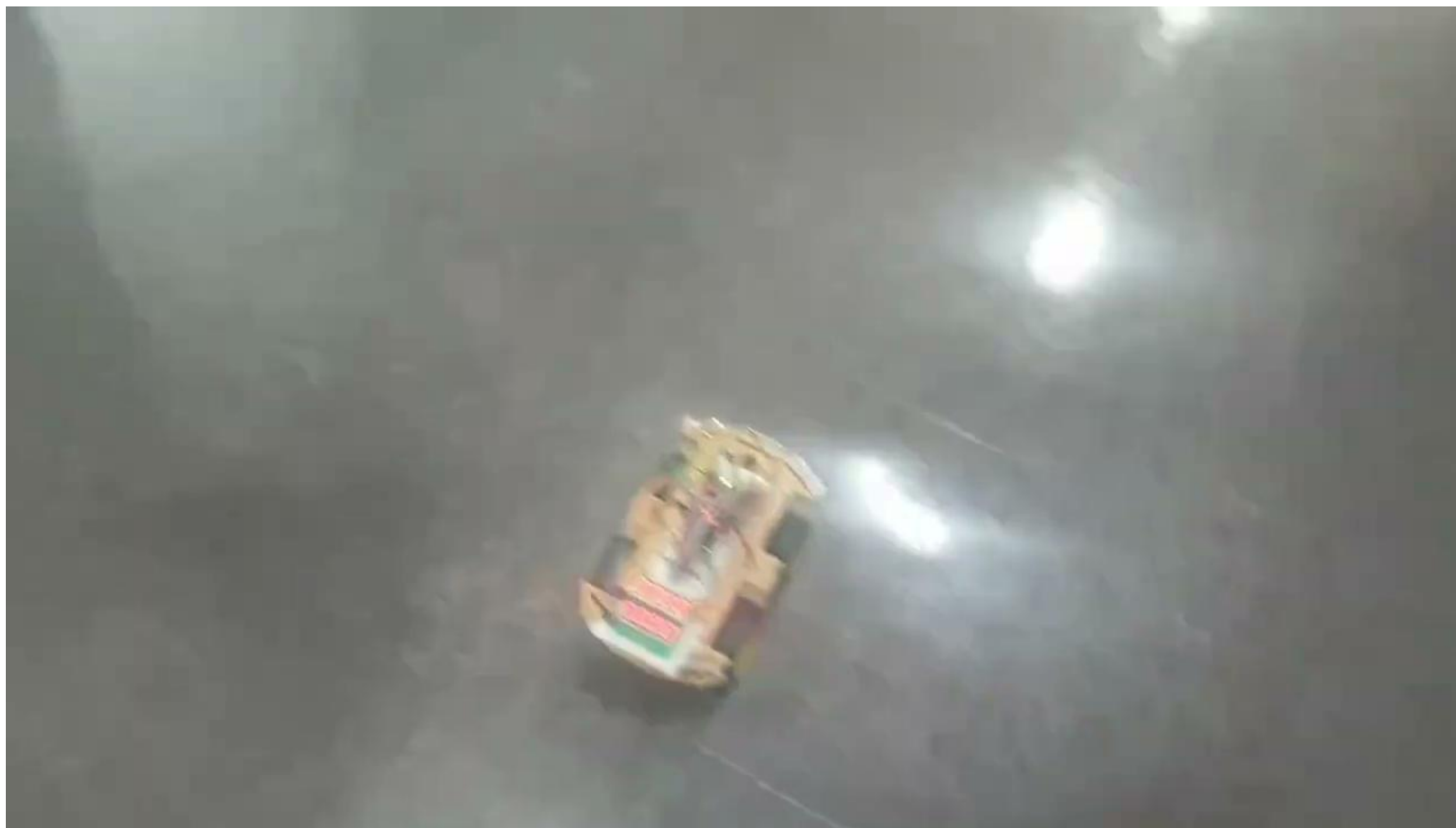
3. 작동 영상





작동 영상

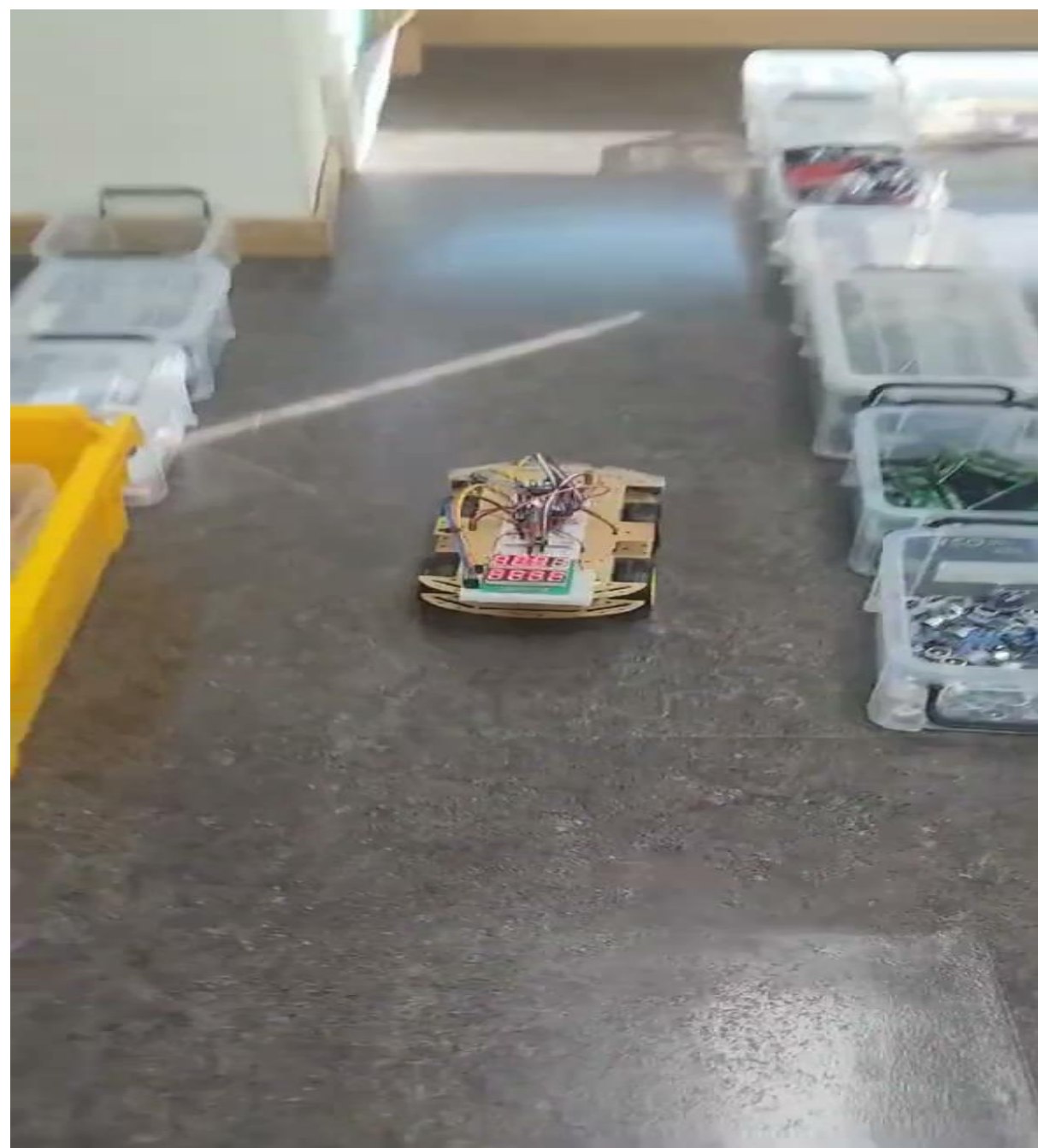
Manual Mode



...

작동 영상

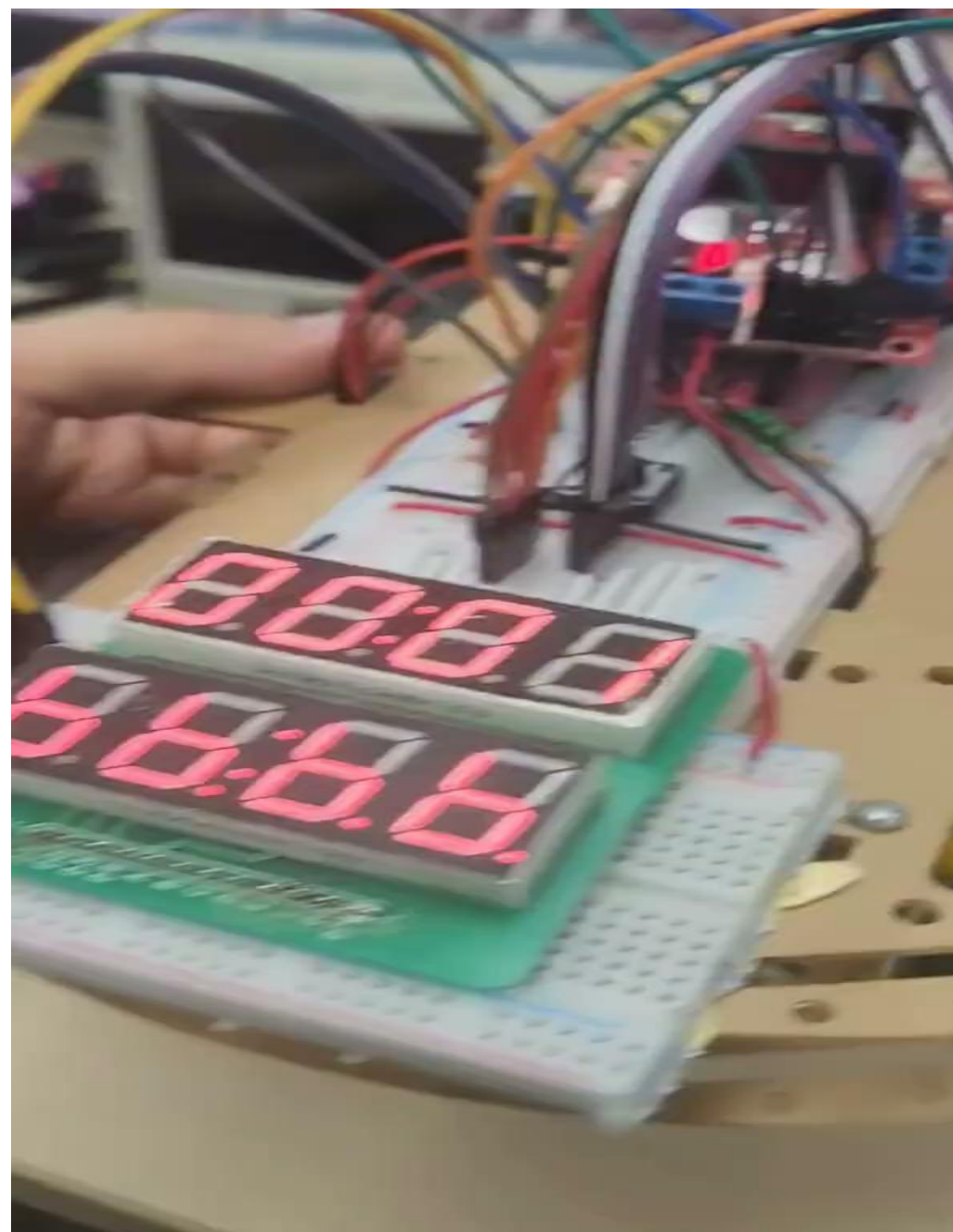
Auto Mode



...

작동 영상

FND





4. 기능 구현

S / W 코드

```
int main(void)
{
    init_led();
    init_timer0();
    init_uart0();
    init_fnd();
    init_button();

    sei();

    init_motor_driver();
    init_timer1_pwm();
    init_ultrasonic();

    stdout = &OUTPUT;

    while (1)
    {
        fp_mode[func_state]();

        if(get_button(BUTTON, BUTTONPIN))
        {
            func_state = func_state == 1 ? 0 : 1;
            if(func_state == MANUAL_MODE) stop();
        }

        if(func_state == AUTO_MODE)
        {
            is_auto = 1;
            distance_check();

            if(sensor_step == 0)
            {
                auto_mode_check();
                auto_mode();
            }
        }
        else if(func_state == MANUAL_MODE)
        {
            is_auto = 0;
            run_stop_watch(sec_count, ms, car_direction);
        }
    }
}
```

```
void distance_check(void)
{
    switch(sensor_step)
    {
        case LEFT_TRIG_PIN:
            make_trigger(LEFT_TRIG_PIN);
            sensor_step = 1;
            break;

        case CENTER_TRIG_PIN:
            if(flag_l)
            {
                flag_l = 0;
                make_trigger(CENTER_TRIG_PIN);
                sensor_step = 2;
            }
            break;

        case RIGHT_TRIG_PIN:
            if(flag_c)
            {
                flag_c = 0;

                make_trigger(RIGHT_TRIG_PIN);
                sensor_step = 3;
            }
            break;

        default:
            if(flag_r)
            {
                flag_r = 0;

                sensor_step = 0;
            }
            break;
    }
}
```

```
void auto_mode_check(void)
{
    printf("L=%d C=%d R=%d\r\n",
        ultrasonic_distance_l,
        ultrasonic_distance_c,
        ultrasonic_distance_r);

    if(hold_count > 0)
    {
        hold_count--;
        return; // 아직 이전 방향 유지
    }

    if(ultrasonic_distance_c < 15)
    {
        car_direction = CAR_BACK;
        hold_count = HOLD_CYCLES_BACK;
        car_status_cnt(car_direction);
    }
    else if(ultrasonic_distance_l < 16)
    {
        car_direction = CAR_RIGHT;
        hold_count = HOLD_CYCLES_TURN;
        car_status_cnt(car_direction);
    }
    else if(ultrasonic_distance_r < 14)
    {
        car_direction = CAR_LEFT;
        hold_count = HOLD_CYCLES_TURN;
        car_status_cnt(car_direction);
    }
    else
    {
        car_direction = CAR_FORWARD;
        car_status_cnt(car_direction);
    }
}
```



S / W 코드

```
void manual_mode(void)
{
    switch(data)
    {
        case 'F':
        case 'f':
            forward(500); // speed를 넘겨줌. 4us * 500 = 0.002sec(2ms)
            break;

        case 'B':
        case 'b':
            backward(500);
            break;

        case 'L':
        case 'l':
            turn_left(700); // 2.8ms
            break;

        case 'R':
        case 'r':
            turn_right(700);
            break;

        case 'S':
        case 's':
            stop();
            break;

        default:
            break;
    }
}
```

```
#include "uart1.h"

extern void UART0_transmit(uint8_t data);

volatile uint8_t bt_data;

void init_uart1(void);
void UART1_transmit(uint8_t data);

ISR(USART1_RX_vect) {
    bt_data = UDR1;
    UART0_transmit(bt_data)
}

void init_uart1(void) {
    UBRR1H = 0x00;
    UBRR1L = 207; // 9600bps
    UCSR1A |= 1 << U2X1; // 2배속 설정 (sampling 8)

    UCSR1B |= 1 << RXEN1 | 1 << TXEN1 | 1 << RXCIE1;
}

void UART1_transmit(uint8_t data)
{
    while(!(UCSR1A & (1 << UDRE1))) {
        ;
    }
    UDR1 = data;
}
```

요소기술 - polling & interrupt 방식

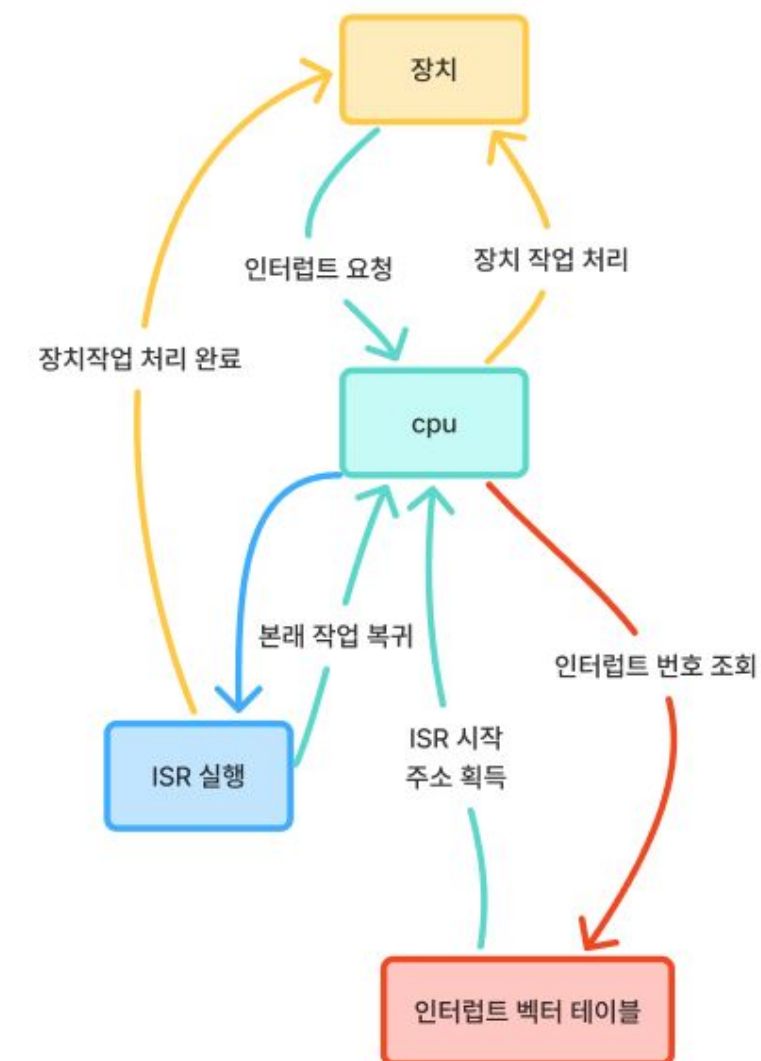
폴링방식 : cpu가 계속 확인하는 방식

예시)



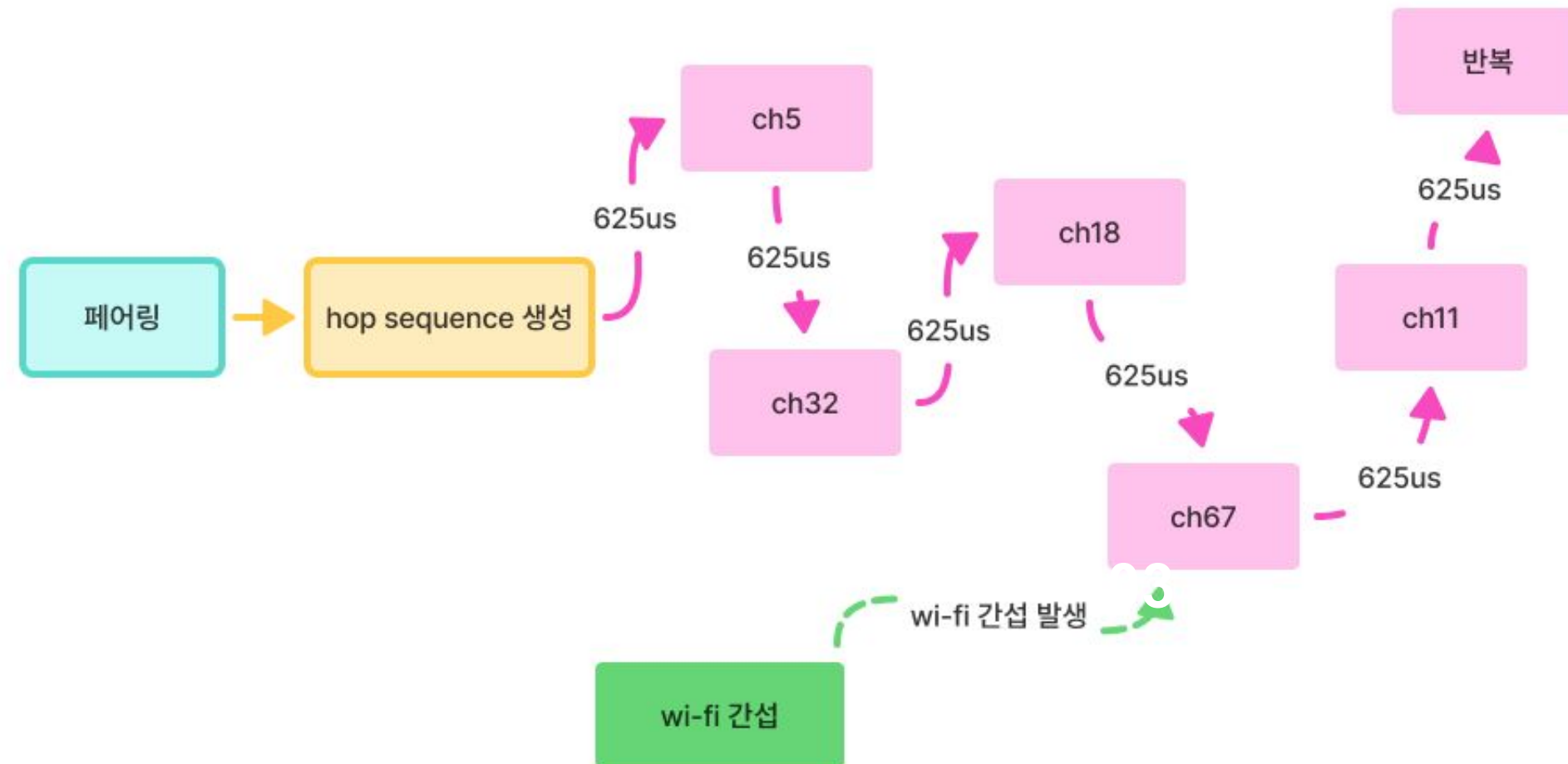
인터럽트 방식 : 문제가 발생했을 때 장치가 cpu에게 알려주는 방식

예시)



요소기술 - Bluetooth통신 주파수 호핑방식

예시)



- 블루투스는 페어링 후 송신기와 수신기가 동일한 Hop Sequence 생성
- 625us마다 채널 변경
- 특정 채널에서 wi-fi 간섭이 발생해도 다음 채널로 이동해서 간섭 영향 감소

요소기술 - PWM(Pulse Width Modulation)

Pulse의 길이(폭)를 바꿔서 신호를 표현한다. 신호의 크기가 클수록 펄스의 폭이 넓어진다.

예시) 펄스 길이 별 전구 세기



장점

- 전력 효율 좋음
- 노이즈 영향 적음
- 회로 간단

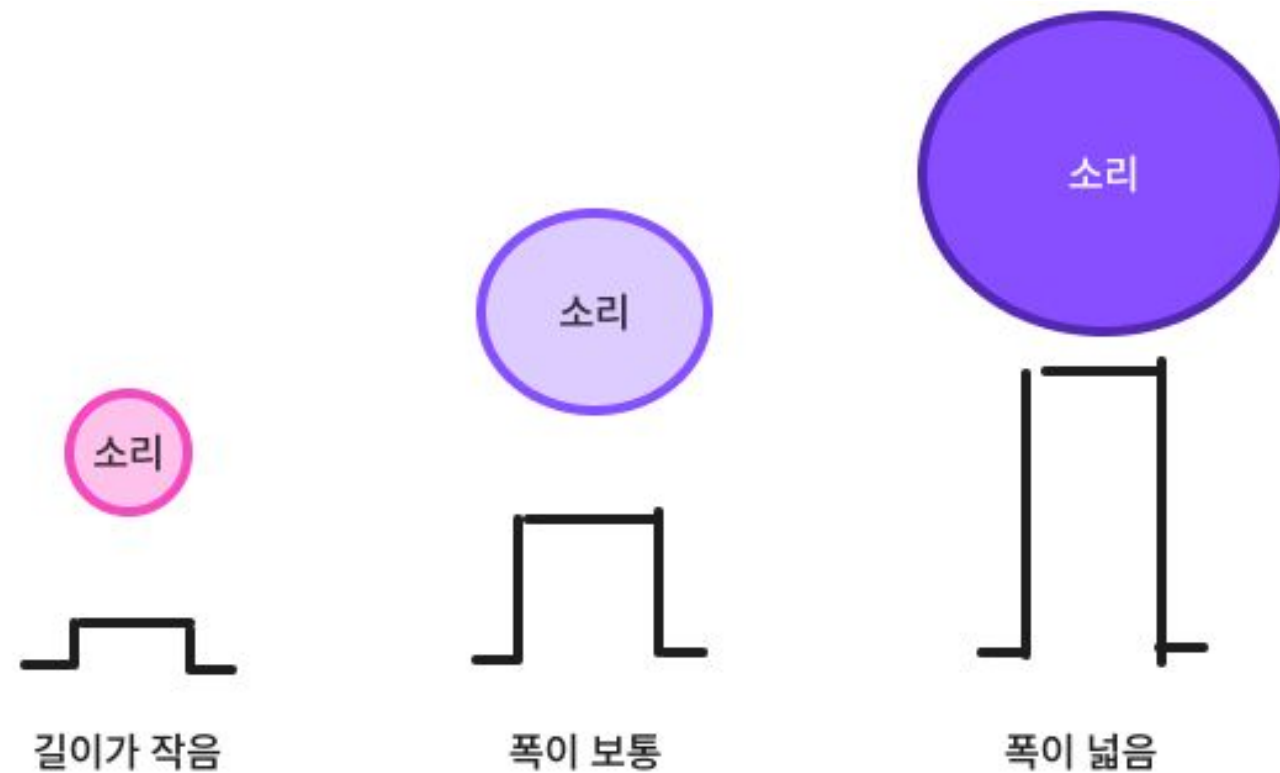
단점

- 실제 신호 크기를 직접 표현하기 어려움

요소기술 - PAM(Pulse Amplitude Modulation)

Pulse의 높이(진폭)를 바꿔서 신호를 표현한다. 신호의 크기가 클수록 펄스의 폭이 넓어진다.

예시) 펄스 높이 별 스피커 세기



장점

- 신호 크기를 그대로 표현 가능
- 오디오 / 통신에 적합

단점

- 노이즈 영향 큼
- 회로가 복잡함
- 전력 효율 낮음



5. 결론



트러블 슈팅 - 김선호

발생한 문제

- 1) uart1를 사용할 때 TX1 RX1에서 PD 2 3 포트를 사용해 연결했던 FND에서 정상 작동을 안하는 문제 발견
- 2) digit을 옮겼더니 pf0123에서 모터를 사용하고있는데 fnd를 초기화할때 한번에 01234567을 초기화하는 것을 발견
이로 인해 모터가 돌아갈 때 fnd가 깜빡거리는 현상 발생

해결 및 방안

- 1) 포트 d를 비우고 c로 옮기고 digit을 pf 4567 pg 0123으로 옮겨 해결
- 2) 해결을 위해 fnd 초기화 시 4567만 따로 초기화 motor도 0123만 분리해서 초기화 시키도록 하여 문제 해결

트러블 슈팅 - 김승환

발생한 문제

- 1) UART1 통신 구성 과정에서 TX1/RX1과 연결된 핀이 FND 제어와 충돌하여, FND 4번째 자리 출력이 비정상적으로 동작하는 문제가 발생 (UART—FND 핀 충돌 문제)
- 2) 마이크로초 단위 타이머 값을 기준으로 주행 제어를 구현하는 과정에서, 특정 주기로 직진과 후진이 반복되는 비정상 동작이 발생 (타이머 측정값 overflow로 인한 역방향 동작)

해결 및 방안

- 1) ATmega의 제한된 핀 자원 내에서 주변장치 기능이 겹치며, 통신 라인과 디스플레이 제어 라인이 동시에 영향을 주는 하드웨어 자원 충돌 문제로 판단. 기존 배선을 재작업하는 리스크가 있다고 판단하여, UART1 대신 UART0 기반으로 통신 구조를 변경. 이 과정에서 UART0가 코드 업로드 라인과 외부 통신 라인을 공용으로 사용하는 제약이 있었기 때문에, 업로드 단계와 동작 단계의 결선 절차를 분리해 운영
- 2) 시간 측정 변수를 int로 선언해 사용했는데, 마이크로초 단위 누적값이 자료형 범위를 초과하면서 overflow가 발생. 그 결과 양수여야 할 값이 순간적으로 음수로 해석되어, 직진 조건에서도 후진 명령이 실행되는 상황 발생. 이에 시간 측정 변수의 자료형을 uint32_t로 변경하여 측정 범위를 충분히 확보하여, 주행 방향이 의도와 다르게 반전되는 문제를 해결.



결론

느낀점

김선호 : 자율주행차 키트를 조립하고 회로 연결, 코딩 후 실행이라는 과정을 하면서 **INT**, 포트와 더불어 **PD 2,3**과 **TX1, RX1**의 상관관계 등 제한사항 등을 알아가며 우회해서 연결하는 재미가 있었습니다. 과정에서 발생한 트러블들을 해결하는 과정에서 **Atmega128A**

를 포함한 하드웨어 장비에서의 데이터**sheet**를 확인하는 습관 등 많은 것을 배운 것 같습니다.

김승환 : 이번 프로젝트를 통해 임베디드 시스템에서는 단순히 기능이 동작하는 것보다, 제한된 자원 안에서 오류 원인을 정확히 분석하고

안정적으로 동작하도록 개선하는 과정이 더 중요하다는 것을 배웠습니다. 특히 **UART** 핀 충돌, 초음파 센서 분기 문제, 타이머 **overflow**

같은 이슈를 해결하면서, 문제를 단순히 수정하는 것이 아니라 구조적으로 원인을 파악하고 적절한 대안을 선택하는 경험을 할 수 있었습니다.



Thank you

